

GRANDOREIRO DFF DLL SIDE-LOADING CHAIN

Source-observed Duplicate Files Finder dependency chain used to load a malicious mingwm10.dll replacement.

HUNTING QUERY

CONFIDENCE: MEDIUM

SOURCE-CONFIRMED BEHAVIOR

retain facts, isolate interpretation

DeviceImageLoadEvents

direct telemetry boundary

HUNT · VALIDATE · TUNE

no automatic malicious verdict

REPORT DATE

20 August 2026

VERSION

v1.0

PRIMARY TELEMETRY

Microsoft Defender XDR ·
DeviceImageLoadEvents

ATT&CK

T1574.002

Evidence-led dossier. Query status remains an untested implementation sketch until executed in the target environment.

INDEPENDENT RESEARCH FORMAT · CONTROLLED UI/UX MASTER



EXECUTIVE ASSESSMENT

Decision-grade summary for detection engineering and SOC operations.

ASSESSMENT

EXECUTIVE SUMMARY

Hunt the two-module sideload chain within one process.

Acronis TRU observed a renamed Duplicate Files Finder executable loading the legitimate dupfdll.dll dependency and then a malicious replacement mingwm10.dll from the same directory. Defender DLL-load telemetry can bind both events to one Windows Process Start Key.

OPERATIONAL VALUE

Specific chain evidence without trusting the randomized process name.

The two modules, same directory and same process identity are materially stronger than a single filename match. Legitimate software may still ship these libraries, so the result remains a Medium-confidence hunt until prevalence and file provenance are measured.

9.1

01

OPERATIONAL VALUE



Medium

02

DETECTION CONFIDENCE



Hunting

03

OPERATIONAL READINESS



SOURCE-CONFIRMED FACTS

Observed behavior

- The campaign was observed in May 2026 and published on 19 August 2026.
- The legitimate Duplicate Files Finder executable was renamed to a randomized filename.
- The executable loaded dupfdll.dll and then a malicious mingwm10.dll replacement.
- Acronis reports both dependencies in the same DLL side-loading chain.

ANALYTICAL ASSESSMENT

What the evidence supports

- The randomized executable name should not be used as a detection condition.
- A shared process identity and co-located module folder retain the source-observed relationship.
- Hash, signature and version metadata are triage evidence, not query predicates.

DETECTION ASSUMPTIONS

What must be validated

- MDE DLL-load telemetry records both modules.
- InitiatingProcessUniqueld is populated and represents the same loading process.
- FolderPath and InitiatingProcessFolderPath use comparable normalization.

Confidence boundary

The source does not establish a durable DFF executable name, signer condition, standard installation path or legitimate prevalence. A match does not prove malicious module content, payload execution, C2 activity or Grandoreiro attribution.



THREAT MECHANICS

Source-observed chain and explicit observability boundaries.

ATTACK FLOW

01

Archive delivery

An invoice-themed ZIP contains decoys and hidden content.

02

Renamed loader

Duplicate Files Finder is renamed to obscure its identity.

03

First dependency

The process loads legitimate dupfdll.dll.

04

Sideload

dupfdll.dll resolves co-located mingwm10.dll.

05

Loader activity

The malicious replacement begins anti-analysis and staging.

Observable core

Two DLL-load events tied to one process identity: dupfdll.dll and mingwm10.dll, co-located with the initiating executable.

Non-observable boundary

Module intent, successful malicious code execution, hidden-window state, decrypted strings, C2 response, second-stage delivery and banking theft.

DETECTION HYPOTHESIS

The source-observed dependency pair should share one process identity.

If an attacker abuses the Duplicate Files Finder chain, DeviceImageLoadEvents should show one process loading both dupfdll.dll and mingwm10.dll from its own directory. Legitimate software ownership, process version metadata, module hashes, signatures, prevalence and acquisition origin distinguish benign packaged dependencies from the campaign.

REQUIRED TELEMETRY

DeviceImageLoadEvents

- Microsoft Defender for Endpoint DLL-load events.
- FileName and FolderPath for both modules.
- DeviceId and InitiatingProcessUniqueld.

OPTIONAL ENRICHMENT

Context that improves disposition

- Process filename, path, command line and SHA1.
- Original filename, product and company metadata.
- Module hash, signature, file-origin and prevalence context.

UNAVAILABLE HERE

Do not infer

- Module content or malicious intent from load events alone.
- GUI suppression and in-memory anti-analysis logic.
- C2 success, second-stage execution or attribution.



TELEMETRY & EVIDENCE MODEL

Minimum data contract and quality gates for a defensible hunt.

DATA CONTRACT

PRIMARY DATA SOURCE

Microsoft Defender XDR - DeviceImageLoadEvents

DeviceImageLoadEvents records DLL loading events and exposes module path, module hash, process identity, process path, command line and version metadata.

EVIDENCE BOUNDARY

A matching event is a lead, not a verdict.

A matching pair proves that one collected process loaded the two named co-located modules. It does not prove mingwm10.dll is malicious or that Grandoreiro executed.

REQUIRED AND ENRICHMENT FIELD MODEL

CATEGORY	FIELD	PURPOSE	REQUIREMENT
DLL events	DeviceImageLoadEvents	Native MDE image-load telemetry.	REQUIRED
Module identity	FileName, FolderPath	Exact dependency names and location.	REQUIRED
Process key	DeviceId, InitiatingProcessUniqueId	Strong within-host process correlation.	REQUIRED
Event identity	Timestamp, DeviceName, ReportId	Investigation and event uniqueness.	REQUIRED
Module hash	SHA1	Candidate-file provenance.	OPTIONAL
Process	InitiatingProcessFileName, InitiatingProcessFolderPath	Co-location and triage.	REQUIRED
Process metadata	InitiatingProcessVersionInfoOriginalFileName, ProductName, CompanyName	Resolve the renamed legitimate loader.	OPTIONAL
Execution	InitiatingProcessCommandLine, InitiatingProcessSHA1	Origin and prevalence pivots.	OPTIONAL

Data quality requirements

- Confirm image-load coverage on representative Windows endpoints.
- Verify InitiatingProcessUniqueId population and semantics.
- Check case and path normalization across both events.
- Measure legitimate prevalence of both libraries together.

Known failure conditions

- MDE is absent or DLL loads are not retained.
- One module load is suppressed or missing.
- InitiatingProcessUniqueId is empty.
- The actor changes either dependency or loading method.



DETECTION LOGIC

Primary-platform logic with every condition traceable to evidence.

HUNTING QUERY

DETECTION OBJECTIVE

Find the reported co-located DFF sideload dependency pair.

Correlate dupfdll.dll and mingwm10.dll loads by DeviceId and Windows Process Start Key, then require both DLLs and the initiating executable to share one folder.

ANALYTICAL LOGIC

Direct filters and investigable output.

Two source-derived module filters, one documented strong process key and direct folder equality. The innerunique join is justified by the two-event dependency chain. No process-name filter, threshold, rarity, score or broad time window.

KQL · UNTESTED HUNT

```
// Hunting query - Untested implementation sketch
// Primary platform: Microsoft Defender XDR / Advanced Hunting
// Required telemetry: Microsoft Defender for Endpoint DLL load events on Windows
// Evidence: Acronis TRU observed a renamed Duplicate Files Finder executable loading
// dupfdll.dll and then a malicious mingwm10.dll replacement from the same directory.
// The process name is not filtered because the campaign renames the executable.
let DffDependencyLoads =
    DeviceImageLoadEvents
    | where FileName =~ "dupfdll.dll"
    | where isnotempty(InitiatingProcessUniqueId)
    | project
        DeviceId,
        InitiatingProcessUniqueId,
        DffDependencyTimestamp = Timestamp,
        DffDependencyFolder = FolderPath;
DeviceImageLoadEvents
| where FileName =~ "mingwm10.dll"
| where isnotempty(InitiatingProcessUniqueId)
| join kind=innerunique DffDependencyLoads on DeviceId, InitiatingProcessUniqueId
| where FolderPath =~ DffDependencyFolder
| where FolderPath =~ InitiatingProcessFolderPath
| project
    Timestamp, DffDependencyTimestamp, DeviceId, DeviceName, ReportId,
    InitiatingProcessUniqueId, InitiatingProcessFileName,
    InitiatingProcessFolderPath, InitiatingProcessSHA1,
    InitiatingProcessVersionInfoOriginalFileName,
    MaliciousCandidateFileName = FileName,
    MaliciousCandidateFolder = FolderPath, MaliciousCandidateSHA1 = SHA1,
    DffDependencyFolder
| order by Timestamp desc
```

STATUS	THRESHOLD	JOIN / SEQUENCE	PRIMARY KEY	DISPOSITION
Untested hunt	None	None	DeviceId + InitiatingProcessUniqueld	Manual triage

What a match means

One process loaded the exact two-module co-located dependency chain reported in the campaign and requires immediate file-provenance review.

What a match does not prove

Malicious module content, code execution success, anti-analysis, C2 communication, financial theft or Grandoreiro attribution.



TRIAGE & INVESTIGATION

Evidence collection before escalation.

SOC WORKFLOW

NUMBERED WORKFLOW

- 01

Confirm both raw events.
Verify device, Process Start Key, timestamps, paths and ReportId values.
- 02

Resolve the loader.
Review original filename, product, company, SHA1, signature and acquisition origin.
- 03

Collect both DLLs.
Compare hashes, signatures, compile metadata and prevalence.
- 04

Validate co-location.
Confirm loader and both modules existed in the same directory at execution time.
- 05

Review follow-on.
Inspect hidden-window, discovery, DNS-over-HTTPS and outbound TCP/6432 evidence.

Escalation gate

Escalate when the loader is newly acquired or unsigned, mingwm10.dll is untrusted or low-prevalence, the directory is user-writable, or the chain is followed by source-aligned discovery and network activity.

Primary benign overlap

Legacy MinGW-based or Duplicate Files Finder software may legitimately package both DLL names together. Ownership, installer provenance, trusted hashes, signatures and stable deployment paths are required before tuning.

INVESTIGATION PIVOTS

Process
Original filename, product metadata, path and signer.

Modules
Hashes, signatures, first seen and prevalence.

Archive
Zone identifier, parent process and delivery metadata.

Host
Role, user, neighboring alerts and prior executions.

Network
dns.google requests and outbound TCP/6432.

Follow-on
Second-stage file, process and persistence evidence.

DISPOSITION CRITERIA

OUTCOME	MINIMUM EVIDENCE	ACTION
BENIGN	Approved, owned and repeatable activity.	Document and tune narrowly.
SUSPICIOUS	Unknown activity without corroborated malicious follow-on.	Deepen endpoint review.
LIKELY MALICIOUS	Unauthorized match plus coherent file, process, network or control-plane evidence.	Escalate incident response.



FALSE POSITIVES & VALIDATION

Operational controls required before promotion.

QUALITY GATE

FALSE-POSITIVE MATRIX

SCENARIO	WHY IT OVERLAPS	TUNING ACTION
Legitimate DFF installation	The original application may ship local dependencies.	Validate installer source, product metadata, hashes and expected path.
Legacy MinGW application	Older software can use mingwm10.dll legitimately.	Require the full pair, same process and co-location; inventory owners.
Software packaging test	Packaging teams may stage portable application bundles.	Check ticket, package source, signer and test window.

Baseline plan

- Run across the full available image-load retention.
- Review every correlated process and both module hashes.
- Inventory authorized DFF and MinGW software.
- Measure process-key and version-field completeness.

Validation procedure

- Replay a benign two-module bundle in an isolated endpoint.
- Verify the join returns one process-scoped result.
- Execute all three negative controls.
- Classify historical matches with software owners.

TEST CASES

CASE	EXPECTED RESULT
Controlled DFF-compatible bundle with both DLL names	Expected correlated match with one Process Start Key.
Load only dupfdll.dll	No match.
Load only mingwm10.dll	No match.
Load both modules from different folders	No match.

Hunting

Initial query

Candidate

Current status

Production

Measured precision

Optimized

Stable feedback

Success

The positive replay matches, negative controls do not, process identity and paths are complete, and historical overlap is absent or attributable to owned software.

Rejection

Common legitimate software reproduces the complete pair and co-location without a durable provenance discriminator.

Failure

One or both DLL events are missing or cannot be correlated with a populated Process Start Key.



ATT&CK, CONCLUSION & REFERENCES

Coverage statement, residual risk and document control.

CLOSURE

MITRE ATT&CK MAPPING

TECHNIQUE	NAME	COVERAGE STATEMENT
T1574.002	Hijack Execution Flow: DLL Side-Loading	Directly covers a legitimate executable loading a malicious replacement DLL through the Windows dependency search path.

Detection coverage summary

Strong for the source-observed dupfdll.dll plus mingwm10.dll pair inside one co-located process; weak for renamed dependencies, reflective loading, missing image telemetry or a different legitimate loader.

Residual risk

The actor can rename or replace either DLL, alter the dependency chain, load from memory or suppress endpoint telemetry. File creation, signature, archive-origin and network detections remain complementary.

ANALYST CONCLUSION

A specific campaign hunt that preserves the DLL relationship.

Acronis supplies a direct module chain and Microsoft documents a process-scoped correlation key. Requiring both DLLs and co-location avoids reliance on the randomized executable name, but environmental prevalence and file provenance are still unknown. The result therefore remains Hunting rather than Production.

REFERENCES

- Acronis Threat Research Unit, “Grandoreiro goes north: From Brazil to Mexico with a new DLL sideloading campaign,” 19 August 2026; campaign observed May 2026 and telemetry reviewed for June 2026. [https://www\[.\]acronis\[.\]com/en/tru/posts/grandoreiro-goes-north-from-brazil-to-mexico-with-a-new-dll-sideloading-campaign/](https://www[.]acronis[.]com/en/tru/posts/grandoreiro-goes-north-from-brazil-to-mexico-with-a-new-dll-sideloading-campaign/)
- Microsoft Learn, “DeviceImageLoadEvents table in the advanced hunting schema,” updated 11 June 2025; accessed 20 August 2026. [https://learn\[.\]microsoft\[.\]com/en-us/defender-xdr/advanced-hunting-deviceimageloadevents-table](https://learn[.]microsoft[.]com/en-us/defender-xdr/advanced-hunting-deviceimageloadevents-table)
- MITRE ATT&CK, T1574.002, DLL Side-Loading; accessed 20 August 2026. [https://attack\[.\]mitre\[.\]org/techniques/T1574/002/](https://attack[.]mitre[.]org/techniques/T1574/002/)

DOCUMENT CONTROL

VERSION	DATE	STATUS	OWNER
v1.0	20 August 2026	Hunting	Detection Engineering

Publication control

Eight A4 pages. URLs are defanged. No customer data, credentials, HTML source or rendering assets are included in the public repository.